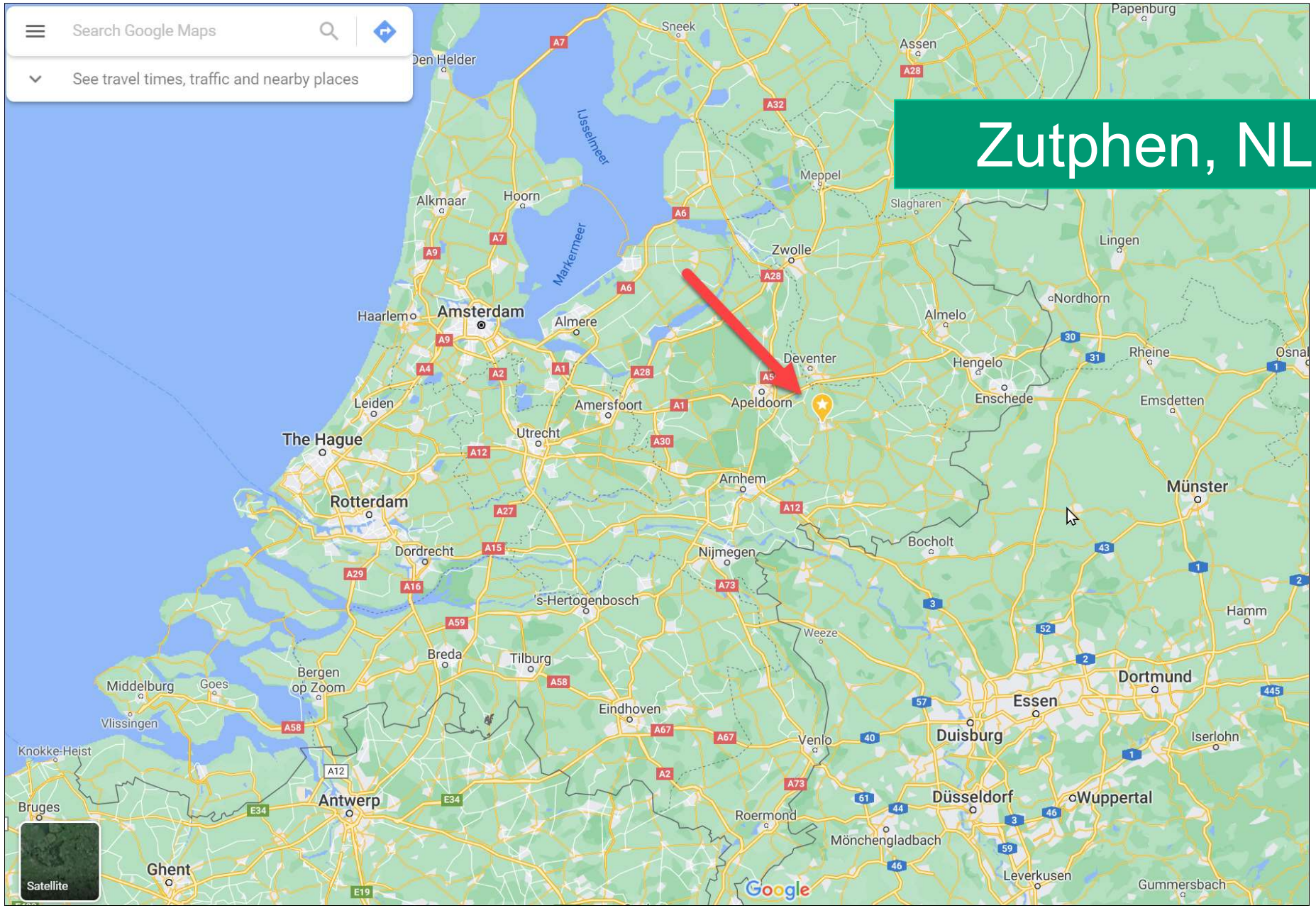
Angular Advanced
Introduction, Architecture

Peter Kassenaar
info@kassenaar.com

Zutphen, NL


Search Google Maps
See travel times, traffic and nearby places
Den Helder
Sneek
Assen
Papenburg
IJsselmeer
Markermeer
Meppel
Slagharen
Alkmaar
Hoorn
Zwolle
Lingen
Nordhorn
Haarlem
Amsterdam
Almere
Almelo
Deventer
Hengelo
Rheine
Enschede
Emsdetten
Leiden
Amersfoort
Apeldoorn
The Hague
Utrecht
Arnhem
Münster
Rotterdam
Bocholt
Dordrecht
Nijmegen
s-Hertogenbosch
Hamm
Weeze
Breda
Tilburg
Bergen op Zoom
Middelburg
Goes
Dortmund
Essen
Vlissingen
Eindhoven
Duisburg
Venlo
Iserlohn
Knokke-Heist
Antwerp
Bruges
Roermond
Düsseldorf
Wuppertal
Mönchengladbach
Ghent
Leverkusen
Gummersbach
Satellite
Google

Peter Kassenaar

- Trainer, author, developer – since 1996
- Specialty: *"Everything JavaScript"*
- JavaScript, ES6, Angular, NodeJS, TypeScript, jQuery, Vue.js, React

www.kassenaar.com

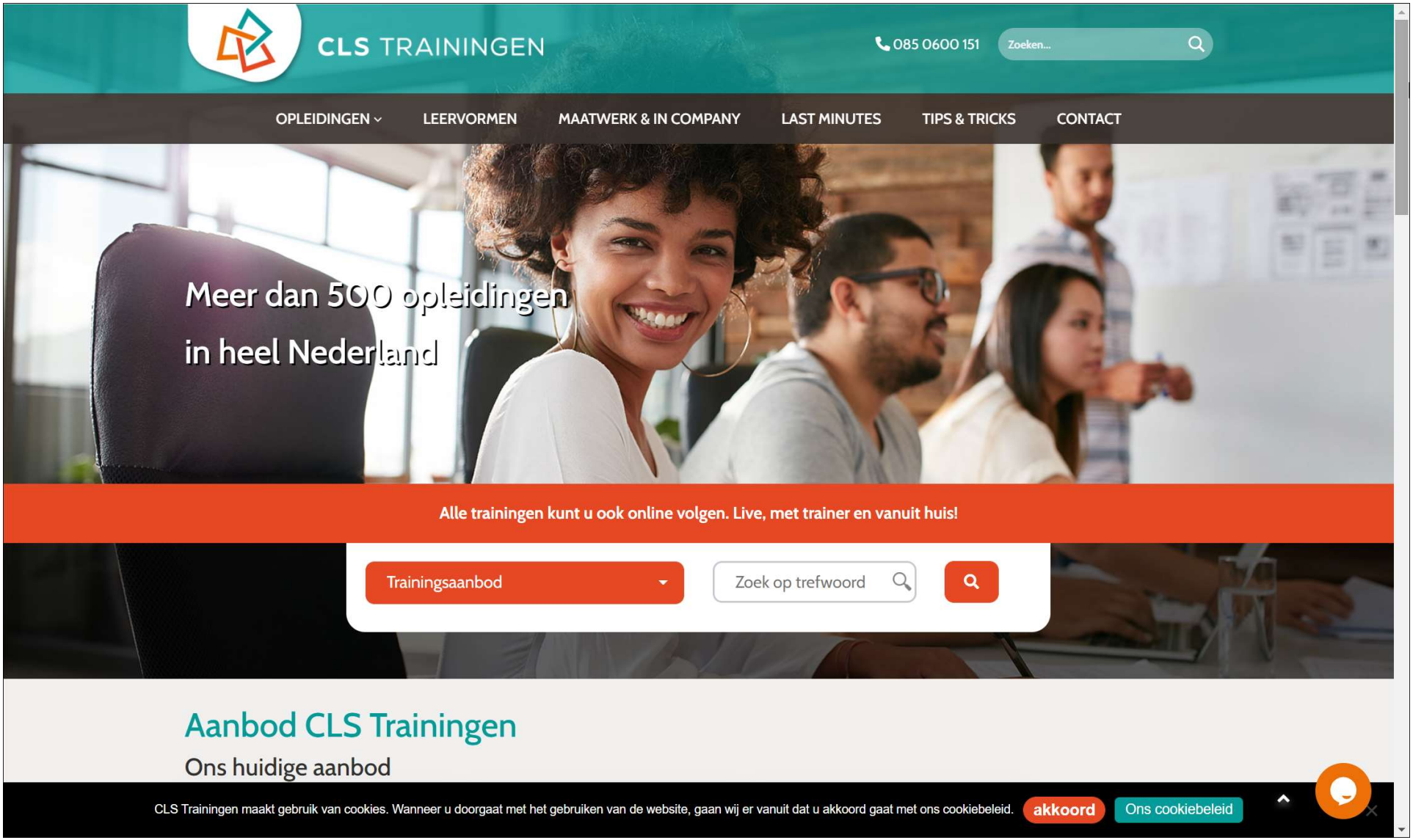
info@kassenaar.com

Twitter: @PeterKassenaar

VANDUUREN MEDIA

ING OHRA euricom A DIMENSION DATA COMPANY sanoma delta lloyd zenito BETERE ZEKERHEID VOOR ONDERNEMERS Atos woonbron OBERON INTERACTIVE ROC West-Brabant the eforum FACTORY

www.angulartraining.nl


CLS TRAININGEN
085 0600 151
Zoeken...
OPLEIDINGEN
LEERVORMEN
MAATWERK & IN COMPANY
LAST MINUTES
TIPS & TRICKS
CONTACT
Meer dan 500 opleidingen
in heel Nederland
Alle trainingen kunt u ook online volgen. Live, met trainer en vanuit huis!
Trainingsaanbod
Zoek op trefwoord
Aanbod CLS Trainingen
Ons huidige aanbod
CLS Trainingen maakt gebruik van cookies. Wanneer u doorgaat met het gebruiken van de website, gaan wij er vanuit dat u akkoord gaat met ons cookiebeleid.
akkoord
Ons cookiebeleid

https://cls.nl/

CLS TRAININGEN

085 0600 151

Angular Advanced

OPLEIDINGEN LEERVORMEN MAATWERK & IN COMPANY LAST MINUTES TIPS & TRICKS

Angular is een platform dat het mogelijk maakt om gemakkelijk webapplicaties te bouwen. Angular combineert verklarende templates, dependency injection, end to end tooling en best practices om ontwikkelingsuitdagingen op te lossen. Met behulp van Angular kunnen applicaties worden gebouwd voor zowel mobiel als desktop.

Omschrijving | Inhoud | Plaats en data

Tijdens de cursus komen de volgende onderwerpen uitgebreid aan bod:

- Angular-applicaties met meerdere (feature) modules;
- routing en lazy loading;
- named router outlets;
- authentication en routing guards;
- advanced components – content projection
- unit testing met Jasmine en Karma;
- smart component/view component design pattern;
- state management met @ngrx/store;
- optioneel: Angular en real-time applicaties met Firebase;
- optioneel: PWA's met Angular;
- optioneel: third party-libraries gebruiken (Angular Material, Ionic, PrimeFaces, Angular Maps, etc).

CLS

Angular Advar

DIRECT I

Deze cursus in

- Locaties door he
- Professionele do
- Inclusief certifica
- 9.4 op Springest
- Ook bij u op loca

github.com/PeterKassenaar/123

Search or jump to... / Pull requests Issues Codespaces Marketplace Explore

PeterKassenaar / 123 Public

Pin | Unwatch 1 | Fork 0 | Star 0

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

main | 1 branch | 0 tags | Go to file | Add file | Code

PeterKassenaar Update README.md 5654d83 last week 3 commits

.gitignore	Update .gitignore	last week
LICENSE	Initial commit	last week
README.md	Update README.md	last week

README.md

123

Slides and example code on the training Angular Advanced, december 2022

Links

- General repository with example code: https://github.com/PeterKassenaar/AngularAdvanced
- ..

About

Slides and example code on the training Angular Advanced, december 2022

Readme
MIT license
0 stars
1 watching
0 forks

Releases

No releases published
Create a new release

Packages

No packages published
Publish your first package

© 2022 GitHub, Inc. Terms Privacy Security Status Docs Contact GitHub Pricing API Training Blog About

Octotree

About you…

Introduce yourself shortly

Knowledge of Angular, (mobile/web-) apps?

How long have you worked with Angular yet?

Tell us a little bit about your projects.

What are your expectations of this course?

Agenda

19-20 December 2022 – Mo. - Tue.

~9:00 start

~ 10:30 Break

~12:00 lunch

~ 14:30 Break

~16:00 closing

Material

Software	(Angular + Editor + Browser + libraries)
Handouts	(PDF, Github)
Workshops	(in the presentations)
Websites	(online)

angular.io/

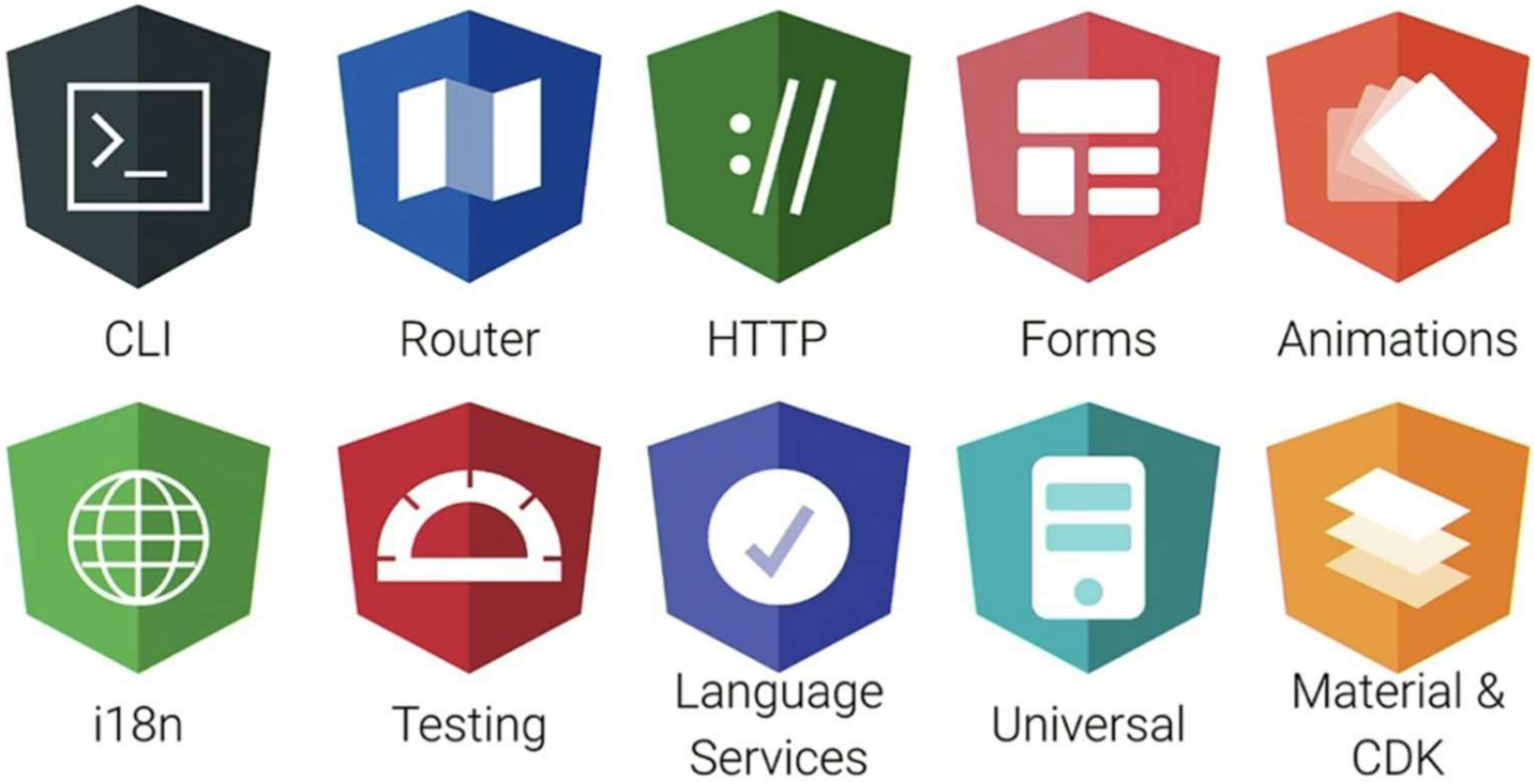
“Advanced”

Broadening?

or...

deepening?

Why devs and enterprises like Angular…



Agenda - 2 days - Thematic

- Day 1: Architecture + Adv. Components
  - Short recap - Angular CLI
  - Composing Applications with multiple modules
  - Routing and lazy loading modules
  - Loading Strategies
- Components – Design patterns
  - Content Projection
  - Smart Components/Dumb components
  - Component reusability

Agenda - 2 days- Thematic

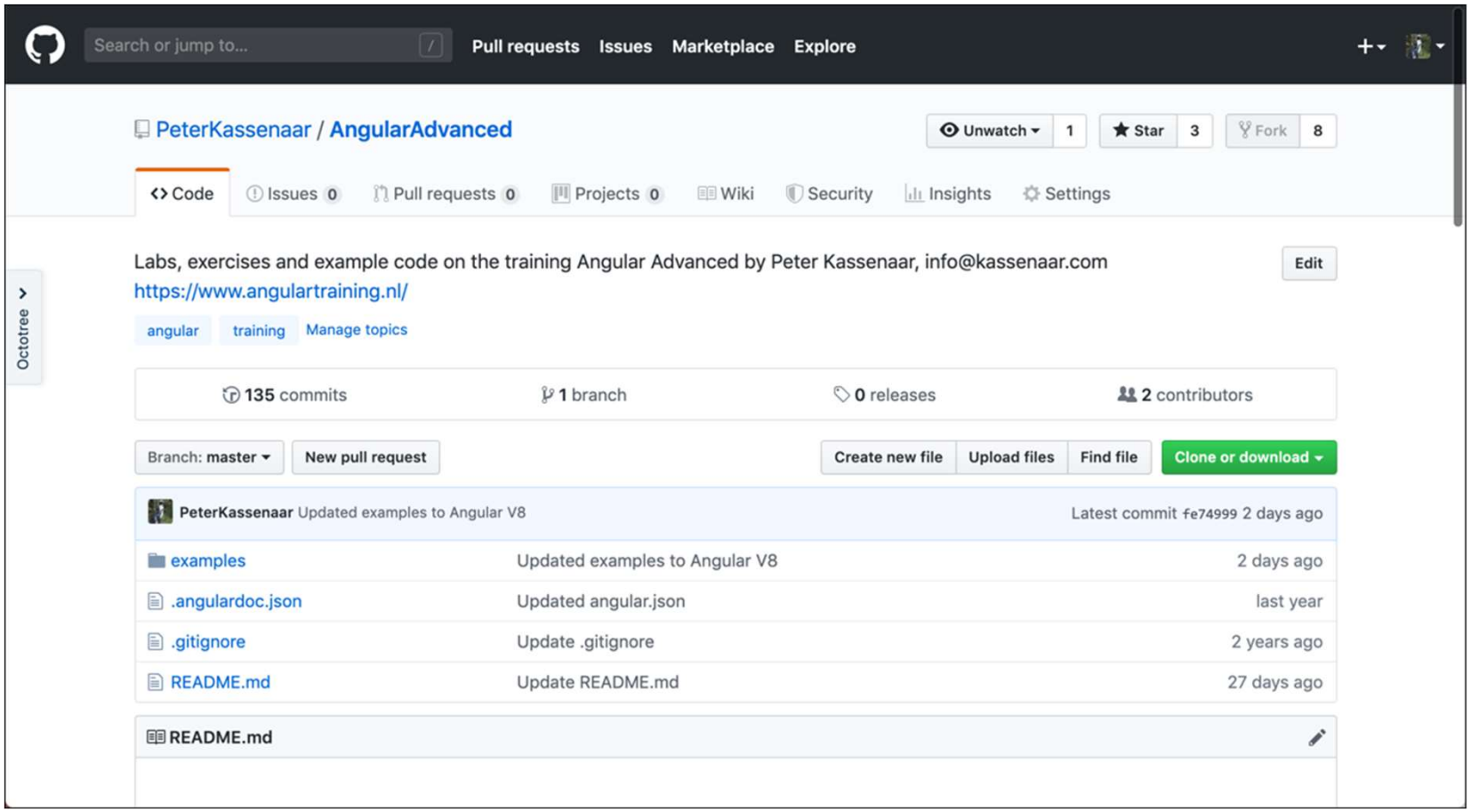
- Day 2: Miscellaneous
  - Observables – pattern + design + operators
  - Angular monorepo's /micro frontends?
  - Introduction - Unit Testing & E2E-testing
  - Publishing Angular libraries to NPM
  - More on Angular Schematics
  - Firebase, PWAs, Elements, …
- Overall : Best practices on coding & architecture

Labs and example code

1. Labs/Exercises
   - In the PDF's in the Github-repo. But: feel free to deviate. Adapt to suit your own needs! (hobby, work, current projects)

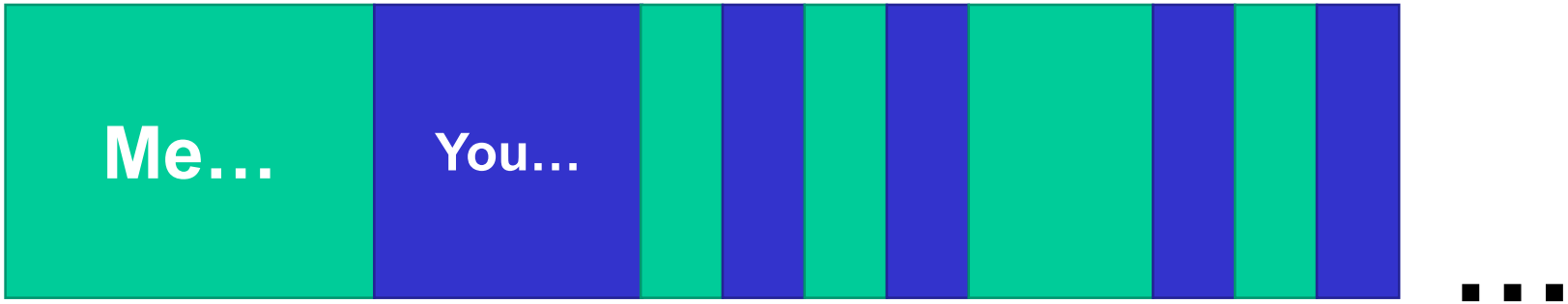
2. Example code
   - Executions of the exercises, small projects (`npm install`, `npm start`)
   - Work in progress – let me know of additions/errors!
   - github.com/PeterKassenaar/AngularAdvanced

Generic ‘Advanced’ Github repo



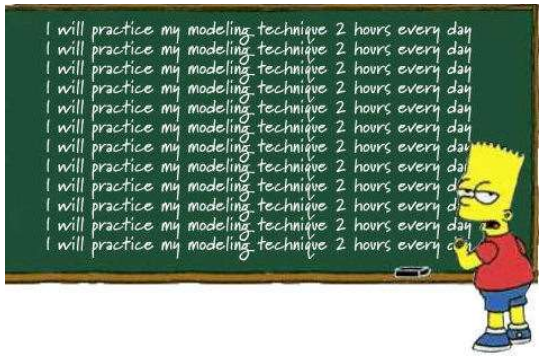
https://github.com/PeterKassenaar/AngularAdvanced

Overall process



On Workshops…

- … are designated with a slide like this 👇
- Are between the talks with theory
- Are NOT written out line-by-line. You have to think for yourself
- The time during the training may be too short to finish all the workshops
  - Do them in your own time
  - At least you know the concepts the workshop is about
  - Choice – we have more to discuss, I let that prevale
- The example code often (but not always!) contains the 'solution' to the workshop. Use it! It's there for you
  - But of course it would be nice if you can work with your own project/data



Questions?

Angular CLI

Scaffold new projects, modules, components via command line…

Introduction
Getting Started
Understanding Angular
Developer Guides
Best Practices
Angular Tools
Tutorials
Release Information
Reference
Conceptual Reference
CLI Command Reference
Overview
Usage Analytics
ng add
ng analytics
ng build

CLI Overview and Command Reference

The Angular CLI is a command-line interface tool that you use to initialize, develop, scaffold, and maintain Angular applications directly from a command shell.

Installing Angular CLI

Major versions of Angular CLI follow the supported major version of Angular, but minor versions can be released separately.

Install the CLI using the `npm` package manager:

```
npm install -g @angular/cli
```

For details about changes between versions, and information about updating from previous releases, see the Releases tab on GitHub: https://github.com/angular/angular-cli/releases

CLI Overview and Command Reference
Installing Angular CLI
Basic workflow
Workspaces and project files
Workspace and project configuration
CLI command-language syntax
Boolean options
Relative paths
Schematics
Command Overview

https://angular.io/cli

We'll be using Angular-CLI this course

- It *is* possible to configure your Angular app by hand
- Using the CLI it's much simpler.
- CLI-options:
  - Scaffolding
  - Generating
  - Testing
  - Building
  - AOT-Compiling
  - …

https://cli.angular.io

Install globally

```
npm install -g @angular/cli
```

CLI is also installed locally in project

```
npm start [--options]
```

https://www.youtube.com/watch?v=wHZe6gGI5RY

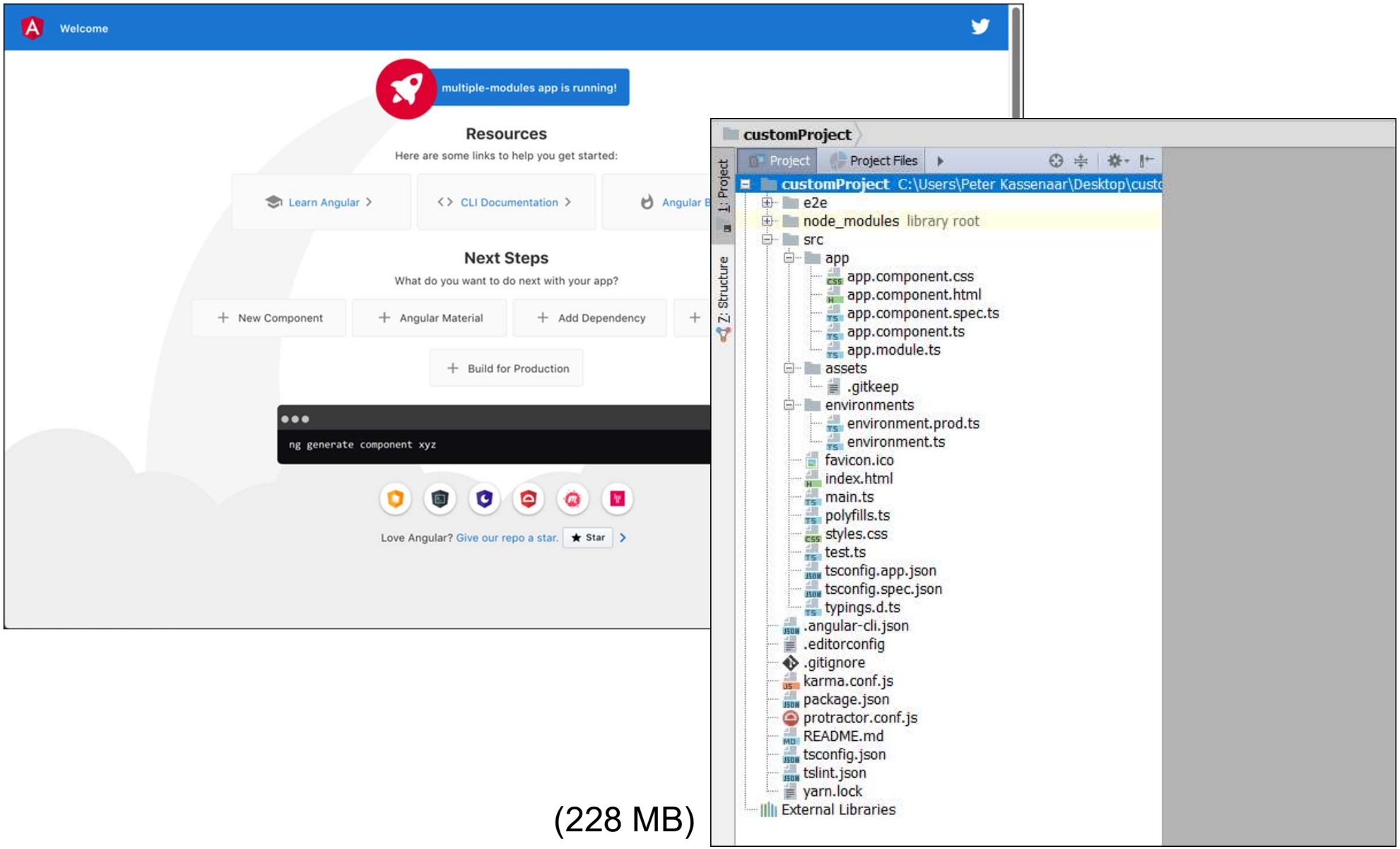
Main commands

`ng new` – create basic app

```
ng new PROJECT_NAME
cd PROJECT_NAME
ng serve // global command, npm start for local command
```

Project is served on `http://localhost:4200`

Default application



(228 MB)

Some CLI tips & tricks

- `ng serve --open` Directly open the compiled project in the browser
- `ng serve --port 4300` Serve project on different port
- `ng serve --ssl` Serve using `https://`
- `ng serve --live-reload false` Do not use live reload
- `ng serve --help` Overview of all other options

More `ng` tooling

- `ng generate <blueprint> --dry-run` Do not write output files
- `ng generate <blueprint> --skip-tests` Do not write spec.ts file
- `ng generate module <name> --routing` add routing to new module

Lots (!) of options

ANGULAR FEATURES DOCS RESOURCES EVENTS BLOG Search

CONFIGURATION
RELEASE INFORMATION
QUICK REFERENCE

Overview
Usage Analytics
ng add
ng analytics
ng build
ng config
ng deploy
ng doc
ng e2e
ng generate
ng help
ng lint
ng new
ng run
ng serve
ng test
ng update
ng version
ng xi18n
API

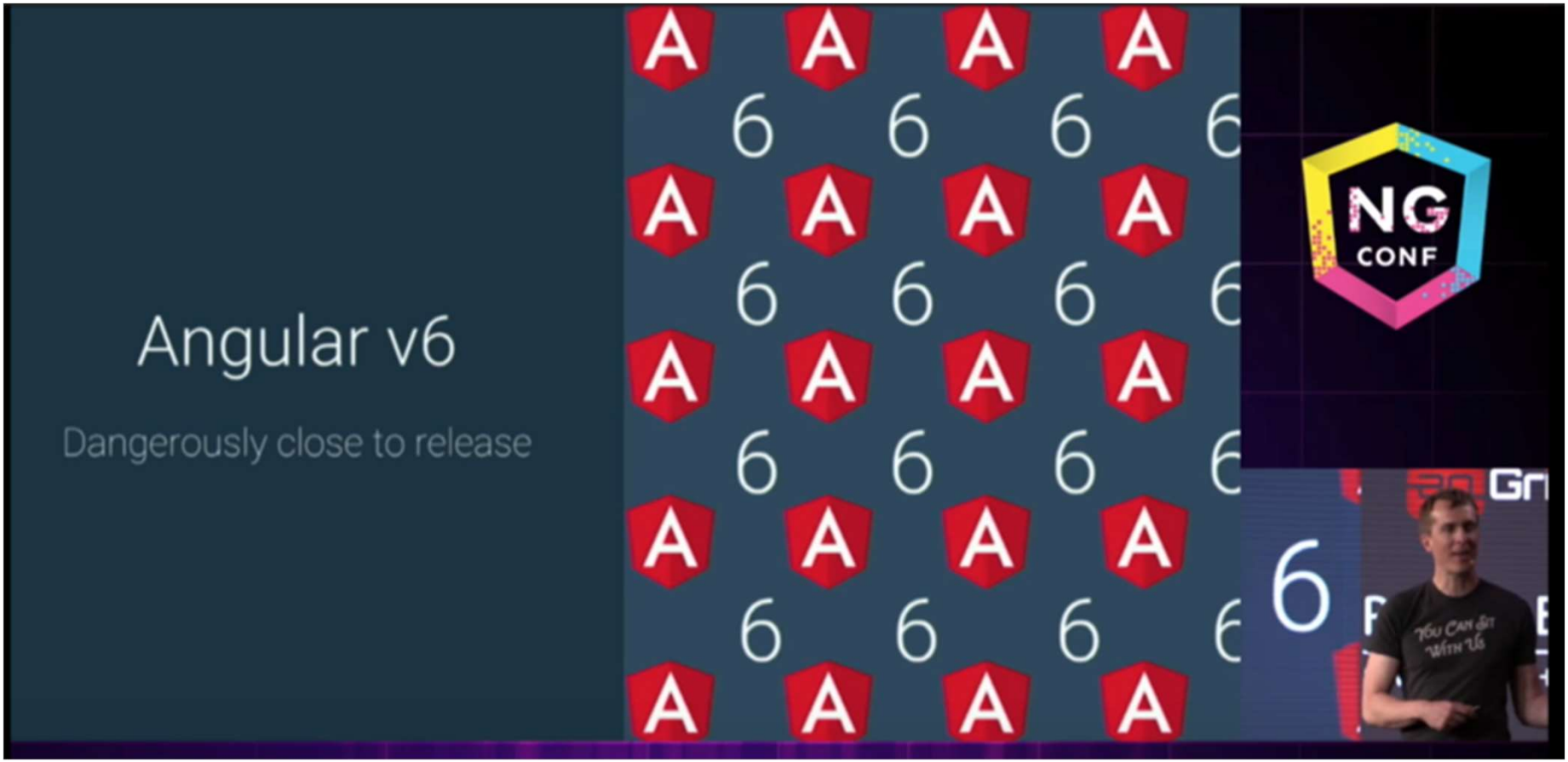
stable (v8.2.14)

Command Overview

COMMAND	ALIAS	DESCRIPTION
add		Adds support for an external library to your project.
analytics		Configures the gathering of Angular CLI usage metrics. See https://v8.angular.io/cli/usage-analytics-gathering.
build	b	Compiles an Angular app into an output directory named dist/ at the given output path. Must be executed from within a workspace directory.
config		Retrieves or sets Angular configuration values in the angular.json file for the workspace.
deploy	d	Invokes the deploy builder for a specified project or for the default project in the workspace.
doc	d	Opens the official Angular documentation (angular.io) in a browser, and searches for a given keyword.
e2e	e	Builds and serves an Angular app, then runs end-to-end tests using Protractor.
generate	g	Generates and/or modifies files based on a schematic.
help		Lists available commands and their short descriptions.
lint	l	Runs linting tools on Angular app code in a given project folder.
new	n	Creates a new workspace and an initial Angular app.
run		Runs an Architect target with an optional custom builder configuration defined in your project.

CLI Overview and Command Reference
Installing Angular CLI
Basic workflow
Workspaces and project files
Workspace and project configuration
CLI command-language syntax
Boolean and enumerated options
Relative paths
Schematics
Building with Bazel
Command Overview

Angular CLI – modern version (May 2018)



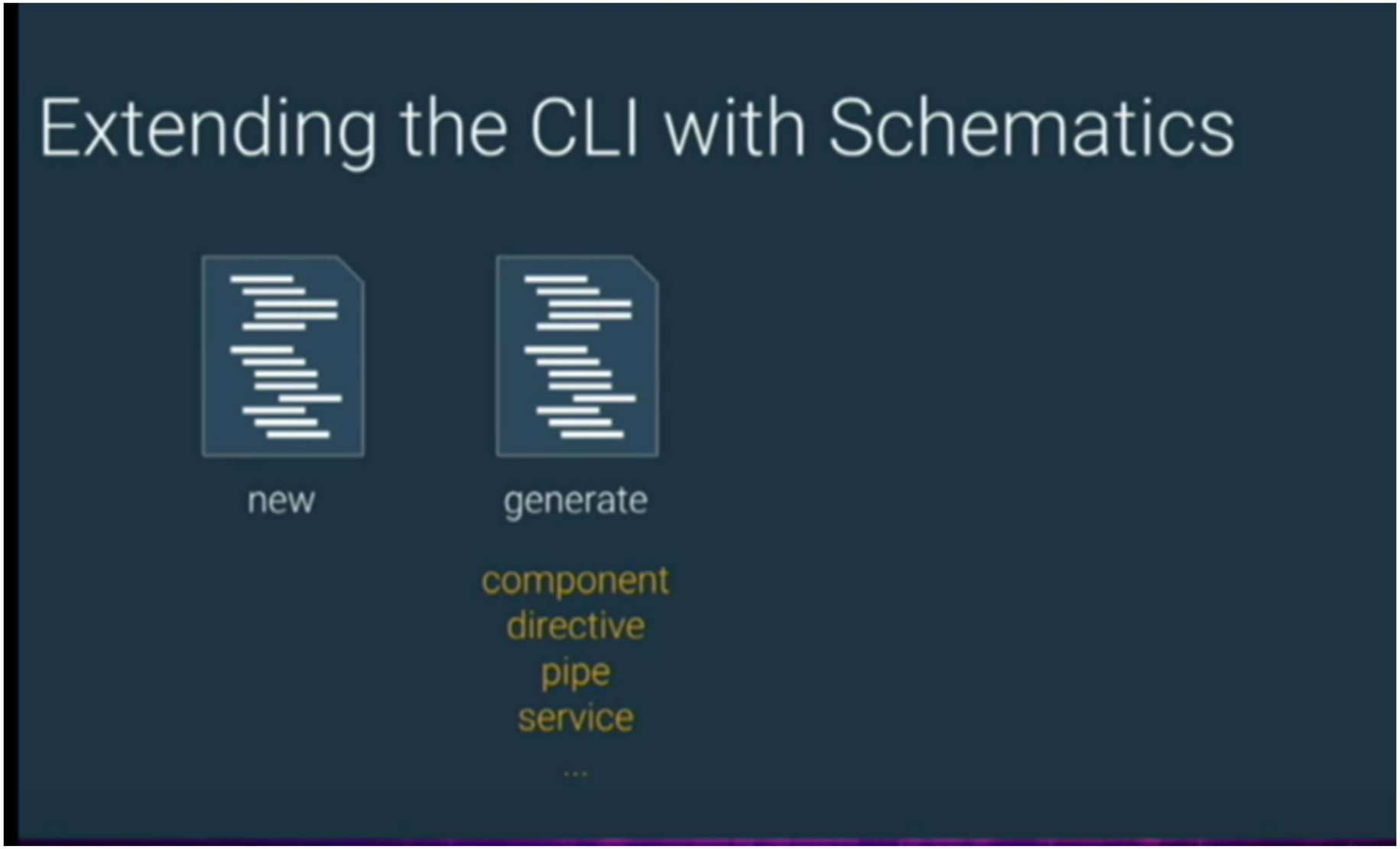
https://www.youtube.com/watch?v=dIxknqPOWms

About version numbering

Aligning Library Releases

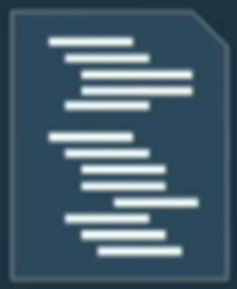
	Today	With v6
Angular	5.2.10	6.0
Material	5.2.4	6.0
CLI	1.7	6.0

New CLI Options



Extending the CLI with Schematics

new



generate

component
directive
pipe
service
...

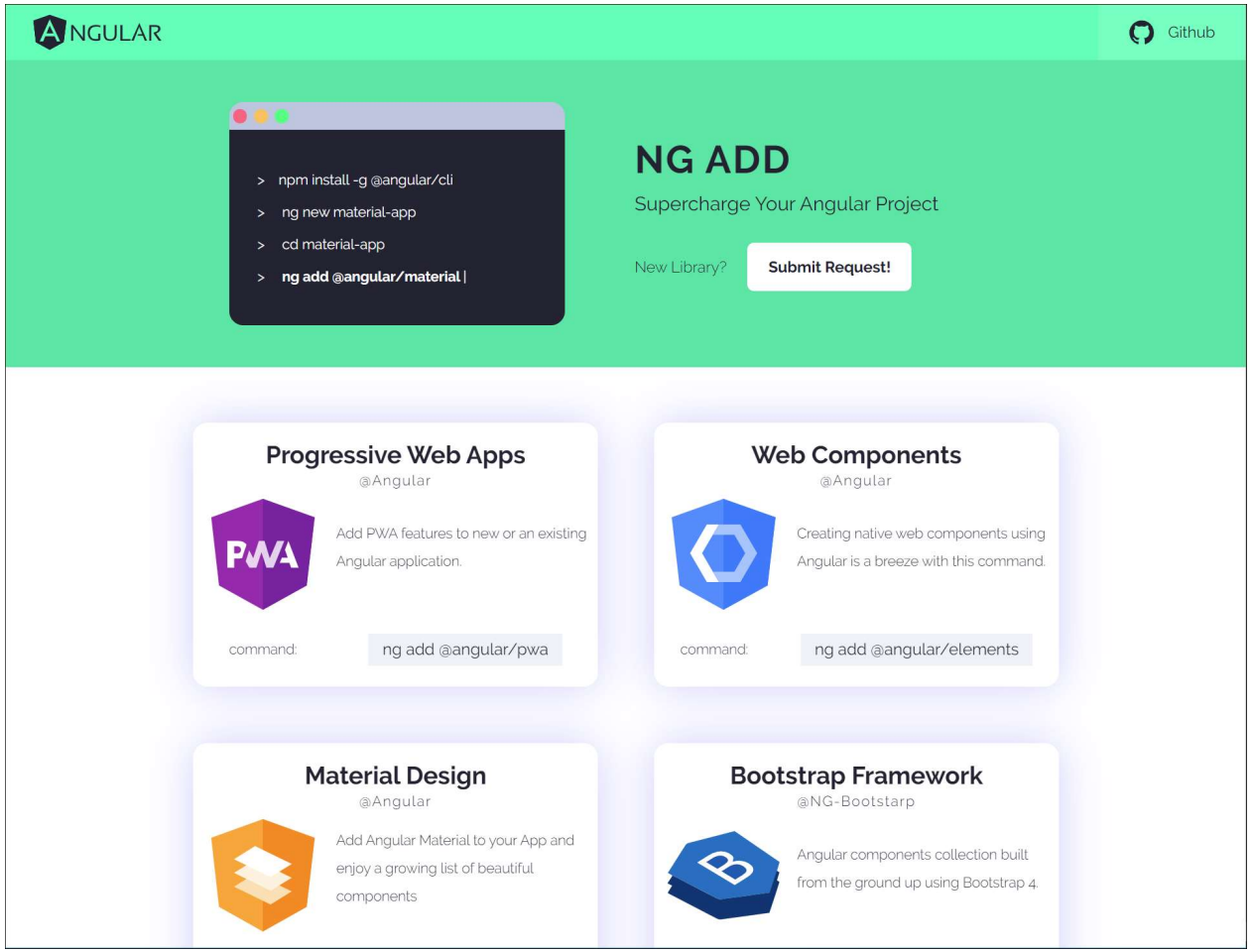
update

add

Info on the Angular CLI keynote

https://www.youtube.com/watch?v=dIxknqPOWms

Ng add overview



https://ng-add.web.app/

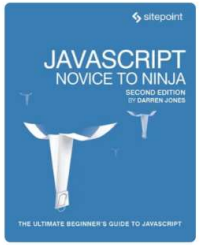
If you want to learn more - background information

Save 55% On Annual Plans! 700+ Assessments, Books and Courses! 07:59:57 Get Now

JavaScript

The Ultimate Angular CLI Reference Guide

AngularJS Tools & Libraries



Jurgen Van de Moere
March 21, 2018

Share

In this article, we'll have a look at what Angular CLI is, what it can do for you, and how it performs some of its magic behind the scenes. Even if you already use Angular CLI, this article can serve as a reference to better understand its inner workings.

Technically, you're not required to use Angular CLI to develop an Angular application, but its many features can highly improve the quality of your code and save you a lot of time along the way.

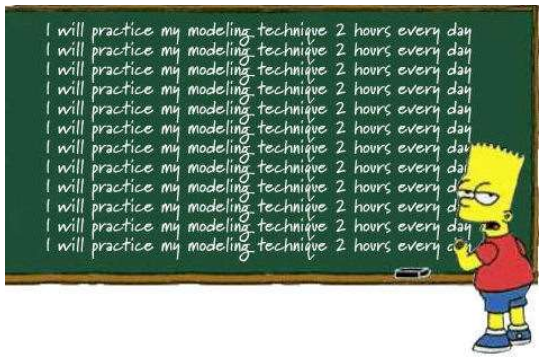
Privacy

https://www.sitepoint.com/ultimate-angular-cli-reference/

Mini Workshop

- Generate a new, blank project with Angular CLI
- Generate a new component or a new service with it
- Add some new CLI packages, using `ng add` for instance
  - `@angular/material`
  - `ngx-bootstrap` (https://valor-software.com/ngx-bootstrap/#/)
  - See how/where they are installed
  - What files are affected?
  - https://ng-add.web.app/
- Study some additional CLI/ `ng` commands at https://angular.io/cli

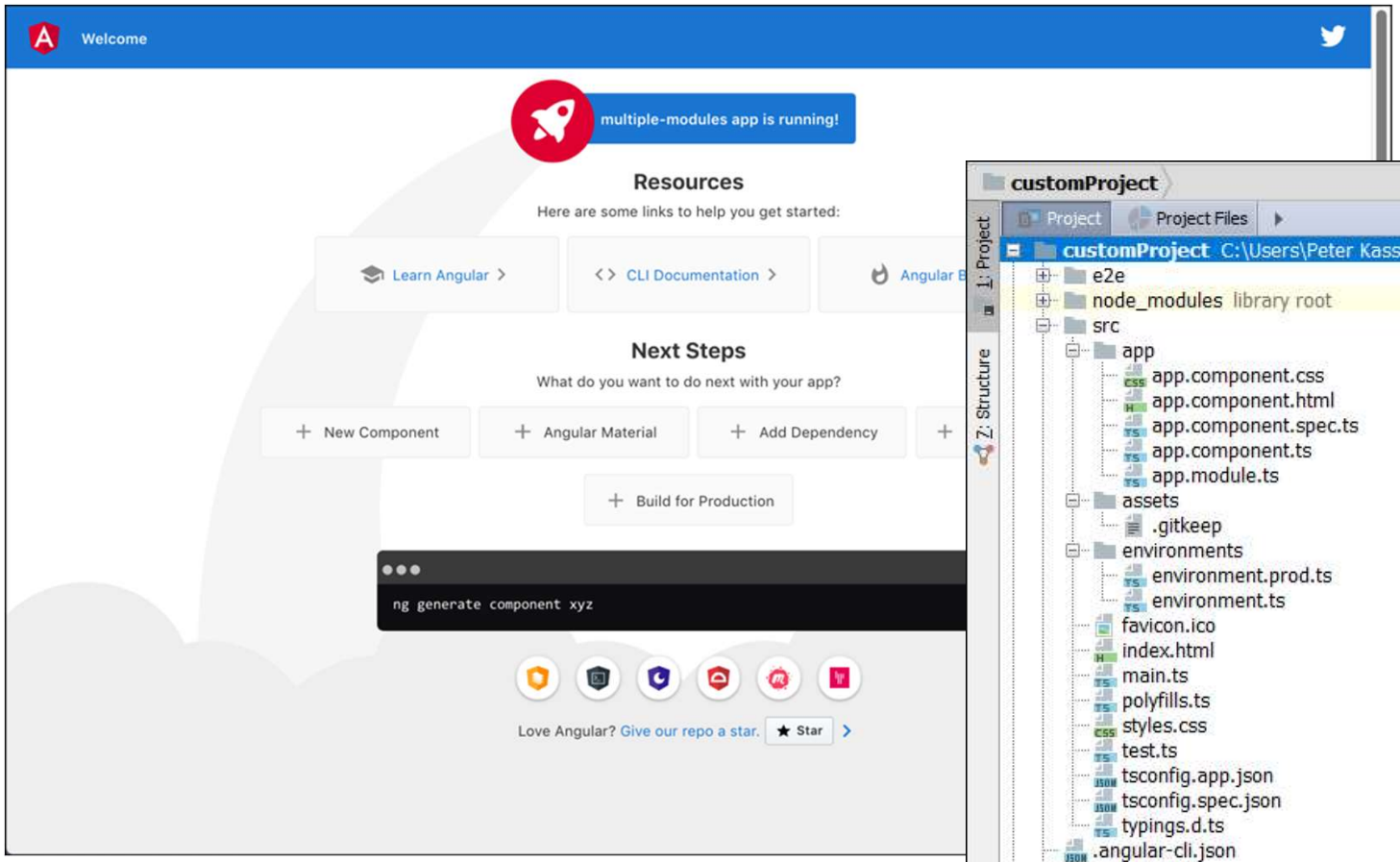
  For instance:
  - What can you generate with `ng generate`?
  - What options are available to `ng build`?
  - What other commands do seem useful to you?

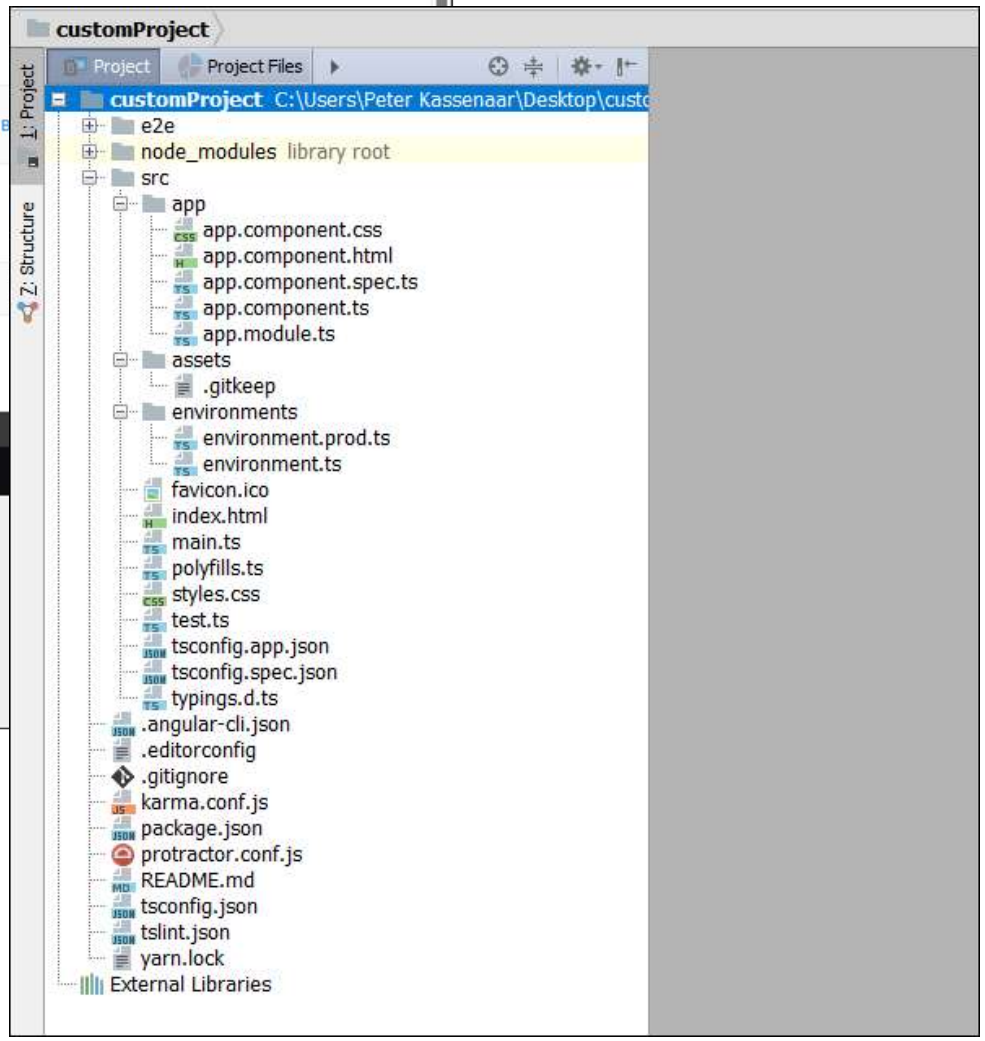


Multiple modules

Splitting your application into separate, reusable modules

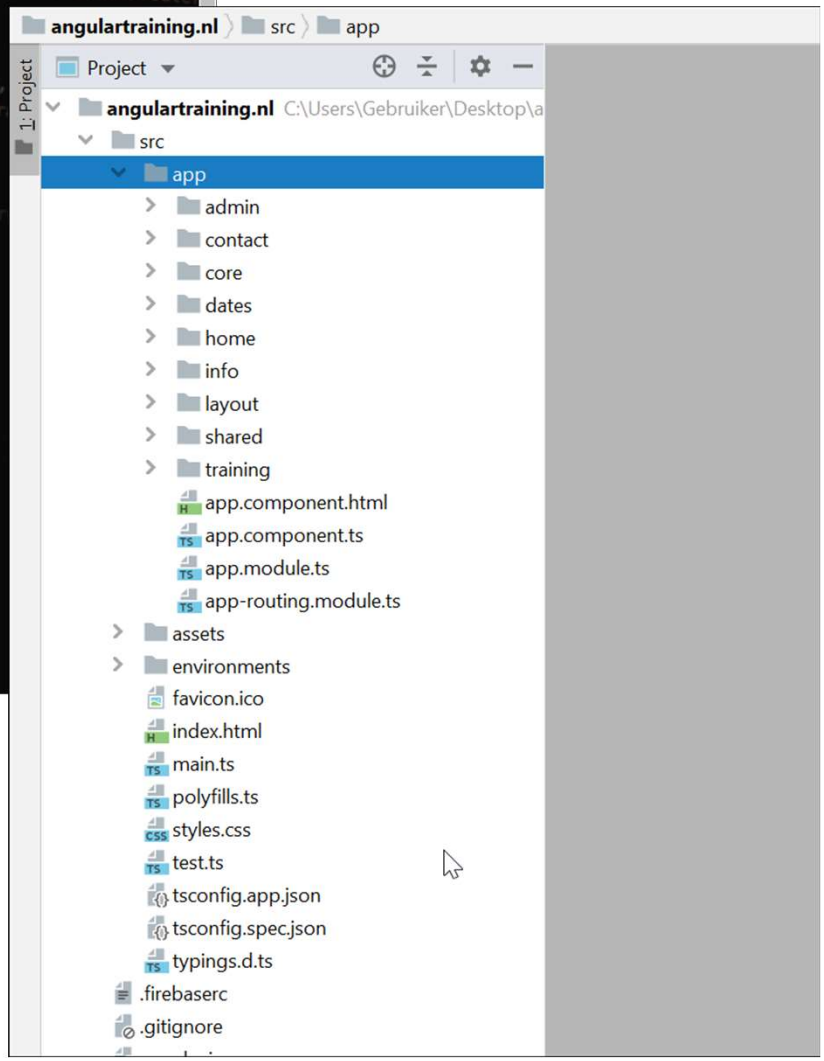
Default application – 1 module





(228 MB)

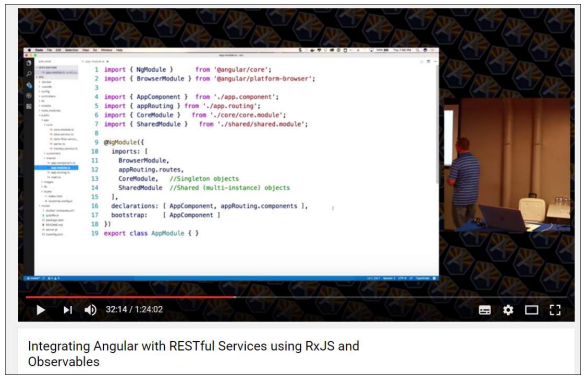
Bigger applications – multiple modules



Angular Modules - naming

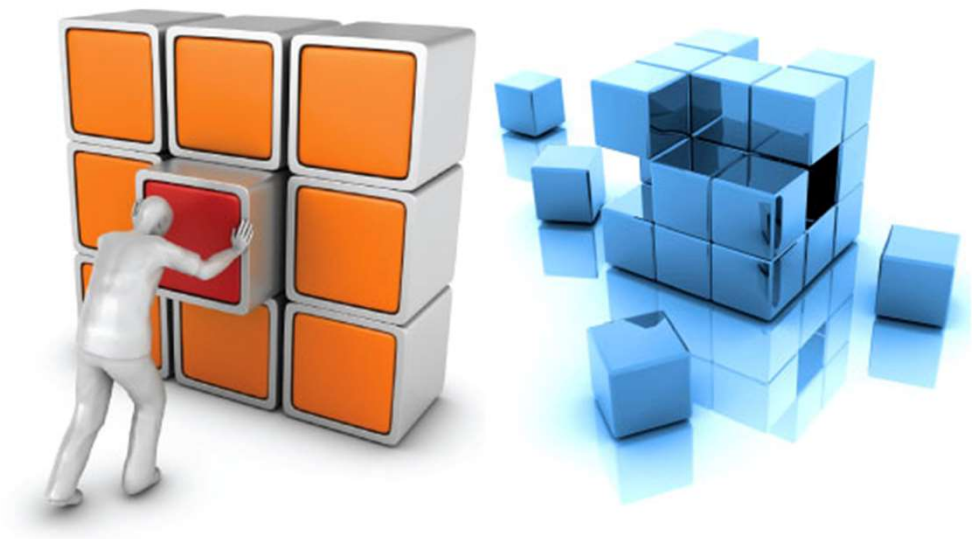
- Divide your app into *logical* and often *reusable* pieces of code
- Keyword : code organization

- Use one `AppModule` - the root of your app
- Use one `CoreModule` - containing all *singletons* in your app
- Use one `SharedModule` - containing all shared resources, possible multiple instances
- Use additional modules *per feature*
- https://www.youtube.com/watch?v=YxK4UW4UfCk



Application – multiple Modules – why?

- *Reuse* of Components, Pipes, Routes and Services etc. over different apps
- *Wrap* each set of logical related components, services, etc. in its own module.



Since Angular 15 – Standalone Components

- Applications and components without an `ngModule`

"Standalone components provide a simplified way to build Angular applications. Standalone components, directives, and pipes aim to streamline the authoring experience by reducing the need for `NgModules`. Existing applications can optionally and incrementally adopt the new standalone style without any breaking changes."

More info on standalone components

SHARE YOUR EXPERIENCE WITH ANGULAR IN THE STAT

--routing 2/3

ANGULAR DOCS COMMUNITY BLOG

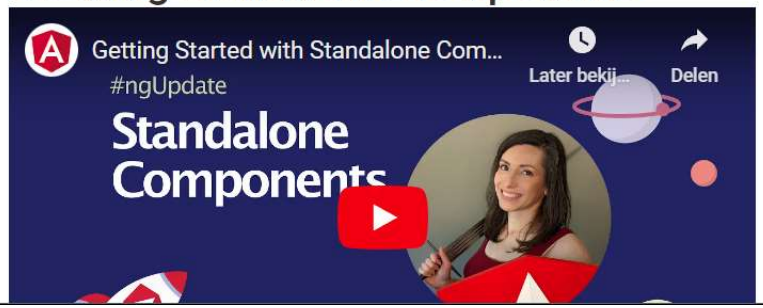
standalone

- Introduction
- Getting started
- Understanding Angular
- Developer guides
  - Overview
  - Standalone components
  - Change detection
  - Routing and navigation
  - Forms
  - HTTP client
  - Image optimization
  - Testing
  - Internationalization
  - Animations
  - Service Workers & PWA

Getting started with standalone components

Standalone components provide a simplified way to build Angular applications. Standalone components, directives, and pipes aim to streamline the authoring experience by reducing the need for `NgModule`s. Existing applications can optionally and incrementally adopt the new standalone style without any breaking changes.

Creating standalone components



- Getting started with standalone components
- Creating standalone components
  - The `standalone` flag and component `imports`
  - Using existing NgModules in a standalone component
- Using standalone components in NgModule-based applications
- Bootstrapping an application using a standalone component
  - Configuring dependency injection
- Routing and lazy-loading
  - Lazy loading a standalone component

https://angular.io/guide/standalone-components

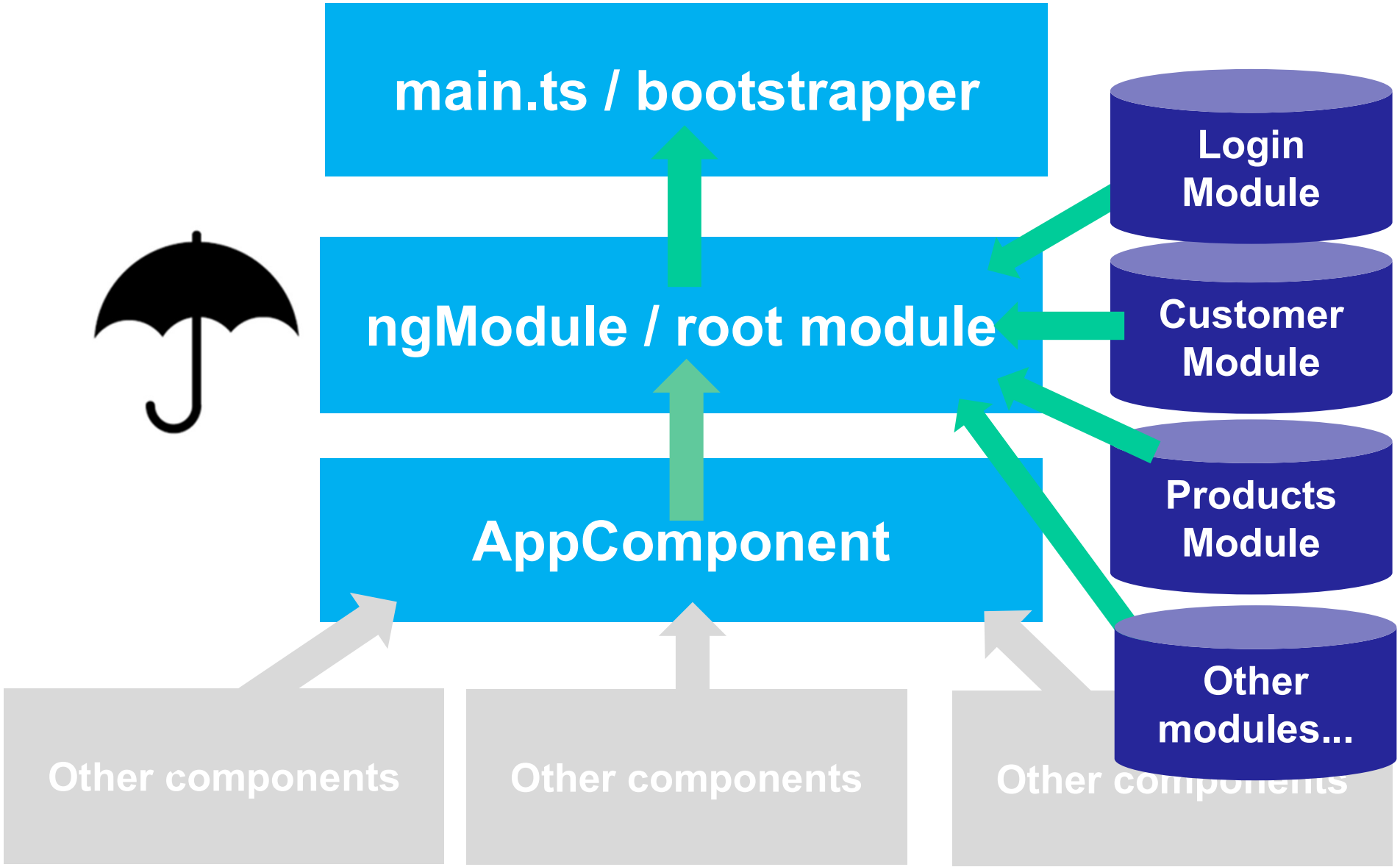
Verdict (personal, opinion!)

We're NOT using standalone components in this course

- Standalone components might come in handy, in new applications from scratch
- Due to confusion, personally I don't see it implemented *'optionally and incrementally'* in existing apps
- It is mostly done (IMO) to mimic Vue, React et all.
- otherwise – little advantages, IMO if you already know Angular

But, if you want more information…

- https://blog.angular.io/angular-v15-is-now-available-df7be7f2f4c8
- https://www.angulararchitects.io/aktuelles/angulars-future-without-ngmodules-lightweight-solutions-on-top-of-standalone-components/
- https://netbasal.com/angular-standalone-components-welcome-to-a-world-without-ngmodule-abd3963e89c5


main.ts / bootstrapper
ngModule / root module
AppComponent
Login Module
Customer Module
Products Module
Other modules...
Other components
Other components
Other components

Steps

1. Create a new module
   - Optional: test first with `--dry-run`
   - `ng generate module customers --dry-run`
2. Create component(s) inside that module
   - Again: test first with `--dry-run`
   - `ng generate component customers --module customers --dry-run`
3. Apply UI, logic, etc. to your component
4. Export your component inside `customers.module.ts`
   - `exports : [CustomersComponent],`
   - Otherwise it can’t be used in other components!
5. Provide new module to `app.module.ts`
   - `imports: [CustomersModule]`

Optional : SharedModule

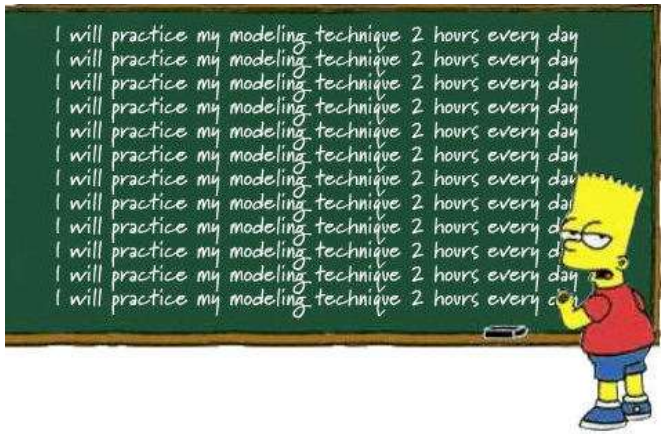
- Reuse components in multiple modules? Use a `SharedModule`
  - `ng g m shared` – shorthand notation
- Create components inside `SharedModule`
- Import `SharedModule` in other modules
- It doesn’t have to be in `AppModule` if you don’t use it directly!
- It *does* not add size to module bundles

Demo

Workshop

- Open `../100-multiple modules.` (`npm install`, `npm start`)
- Create a new module
- Create a new component inside this new module and give it some UI.
- Include the module in the Main Module and show it besides other modules
- Include the Search Component in your own module

- *OR:*
- Add Multiple Modules from scratch to your own application, using the steps described in this module.



How to structure feature modules

242

Why and how to structure Features in Modules in Angular

This might sound pretty basic, but I encounter these challenges over and over in customer projects and it's still an ongoing discussion internally.

A central project goal in a recent Angular project was to design features and UI components for reusability. To achieve this, we need to make sure our code is well isolated and has a simple and clear dependency model.

Prologue: Feature vs. Technical Project Structure

When building small apps and looking at common code samples in the internet a lot of devs (including myself) tend to come up with a project structure like this:

```
MYAPP
  src
    app
      components
        home
          home.component.html
          TS home.component.ts
        user
          user.component.html
```

https://medium.com/@philippbauknecht/why-and-how-to-structure-features-in-modules-in-angular-d5602c6436be